本章路線

這章要回答的核心問題是：作業系統如何在不受信任的程式與有限硬體之間提供方便、效率與隔離？

前置知識。 程式執行、CPU 與記憶體的基本角色、中斷概念。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

學習目標

- 區分 user mode、kernel mode、中斷、例外與 trap
- 說明 programs、processes、files 與 storage hierarchy 的抽象關係
- 判斷 kernel data structure 與環境選擇對正確性及效能的影響

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.1　作業系統做什麼

- OS以受管理抽象協調硬體資源。
- 便利、公平、隔離與利用率是可衝突的不同目標。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

作業系統提供的不是單一功能，而是受管理的抽象

實體資源	程式使用的抽象	OS必須強制的條件
CPU／timer	process/thread執行	誰可跑、何時讓出、誰能改特權狀態
RAM／MMU	address space	映射與權限不可由一般程式任意改他人的
disk／controller	file與block服務	名稱、權限、存取次序與錯誤回報
NIC	socket／訊息	端點、buffer ownership、封包與連線狀態

方便性、利用率、response time、公平、隔離與可靠性是不同目標。例如長batch佔滿CPU可提高utilization，卻讓互動請求久等。需先指定目標與工作負載，不能以「效率高」概括所有指標。

kernel是一般系統的受信任特權核心；shell、GUI、library、daemon多在user space。呼叫library不必然進kernel；換shell也不必換kernel。

policy決定允許什麼，mechanism強制它

例 Alice/Bob共用機器，Bob的程式不可信。policy：Bob不能讀Alice私頁與檔案、不得永久獨占CPU。mechanisms：受保護page table與每次memory access檢查；核心驗證file authority；受保護timer及scheduler入口。

單有應用的if檢查不夠：Bob可跳過該應用直接發請求。單有user/kernel mode也不夠：kernel若替Bob無條件讀敏感檔回傳，仍違反policy。enforcement point必須完整中介相關操作，且其政策／metadata不能由不可信程式改寫。

安全邊界因此包含CPU、記憶體、物件權限及可變共享狀態，不以GUI是否隱藏按鈕判定。已授權reference的撤回又是另一項契約，第11/14章分開分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.2 電腦系統組織

- driver設定controller，DMA搬資料，完成機制交還ownership。
- 同步system call／exception與非同步interrupt分開計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

controller、driver與DMA各負責哪一段

controller是硬體／韌體端的裝置控制邏輯，driver是理解其register/queue/完成狀態的OS軟體。CPU提交命令、裝置搬資料、CPU處理完成是不同事件；完成可用interrupt通知或由polling讀狀態。

固定教學時間：driver設定0–0.1ms，DMA傳輸0.1–4.1ms，完成處理4.1–4.2ms；CPU在中間4ms可執行另一個就緒工作。DMA減少逐byte CPU搬移，不代表設定、cache同步、記憶體頻寬、完成處理都免費。

若block4096B、傳輸1024B/ms，本例transfer=4ms；CPU控制成本0.2ms，總request latency4.2ms。CPU只花0.2ms並不代表應用等待也只有0.2ms。資料必須在裝置／CPU所需時點可見，不能在DMA尚未完成就歸還buffer。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

controller、driver與DMA各負責哪一段 (續)

DMA：request latency=4.2ms，CPU控制成本=0.2ms

藍=CPU設定 / 完成處理；綠=DMA資料傳輸；灰=裝置未傳輸。時間軸按比例。



CPU設定0–0.1ms → DMA完成4.1ms → CPU處理4.1–4.2ms

CPU不逐byte搬移，仍須設定、cache / 排序、完成處理及buffer lifetime；不能提早重用buffer。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

interrupt、同步exception與system call分開分類

事件	相對執行指令	典型後續
裝置完成／timer interrupt	外部非同步	保存被打斷狀態、處理事件、可能排程
page fault／除零exception	由目前指令同步觸發	補頁後重啟、或交錯誤／終止
system call	程式主動同步請求	進受控入口、檢查參數、服務、回user

各ISA對trap/fault/interrupt的術語與返回點不同；不能把所有exception都說成下一指令繼續。裝置interrupt若發生在kernel mode，可能沒有user→kernel transition，但仍需保存／恢復當前execution state及處理重入同步。

interrupt處理完成後可以返回同一thread；system call也可能不阻塞。兩者都不必然發生process context switch。DMA也不必然每block一interrupt，可poll或合批；必須把實際completion規則寫清楚。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.3 電腦系統架構

- SMP、NUMA與cluster的共享關係和失敗模型不同。
- 平行加速與locality成本需以基準時間和明確比例計算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同步與失敗條件來自實際共享關係

架構	通常共享什麼	額外問題
single general-purpose CPU	一組一般指令執行資源	仍可能與DMA／interrupt並行
SMP	對等CPU、共同記憶體	共享資料同步、cache一致性
NUMA shared memory	共同addressable RAM、不同locality	位置影響latency／bandwidth
cluster	獨立節點經network合作	訊息、partial failure、重試／共識前提

core是執行核心，socket/package是封裝，hardware thread是架構暴露的執行context；它們不能不看機器就互換計數。SMP的對等控制與NUMA的成本不均是不同維度，不把所有shared-memory系統都說成UMA。

多CPU不能單獨保證容錯：共享電源、RAM、kernel或storage都可能是共同失效點。failover還需偵測、狀態恢復與防止兩個節點同時擁有排他資源。

NUMA與平行加速的兩個可重算模型

假設一連串無重疊、無cache影響的memory requests，local=100ns、remote=180ns。80%local時平均0.8×100+0.2×180=116ns；移成20%local時為164ns。這是指定比例下的平均單次成本，不直接推得整個程式加速164/116。

基準工作時間100s，其中serial20s、可完美平行80s、無額外overhead：P=4時T=20+80/4=40s，speedup=2.5。一般T(P)=100[s+(1-s)/P]，speedup≤1/s；還要加同步與資料移動成本。

以operation count代替基準時間比例會錯；多CPU也不會自動找出可平行部分。成本比例與可同時做的工作都必須由模型或工作負載提供。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.4 作業系統結構

- multiprogramming利用別人的ready CPU工作覆蓋I/O等待。
- time sharing另需timer、preemption與有進度的scheduler規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

multiprogramming是把I/O等待與別人的CPU工作重疊

固定單CPU、一個I/O裝置、切換免費，A=CPU2→I/O6→CPU2，B=CPU4且無I/O；兩者t=0就緒。

方式	CPU時段	I/O時段	完成所有job	CPU利用率
A做完才跑B	A0–2，idle2–8，A8–10，B10–14	A2–8	14	8/14=4/7
A阻塞便跑B	A0–2，B2–6，idle6–8，A8–10	A2–8	10	8/10=0.8

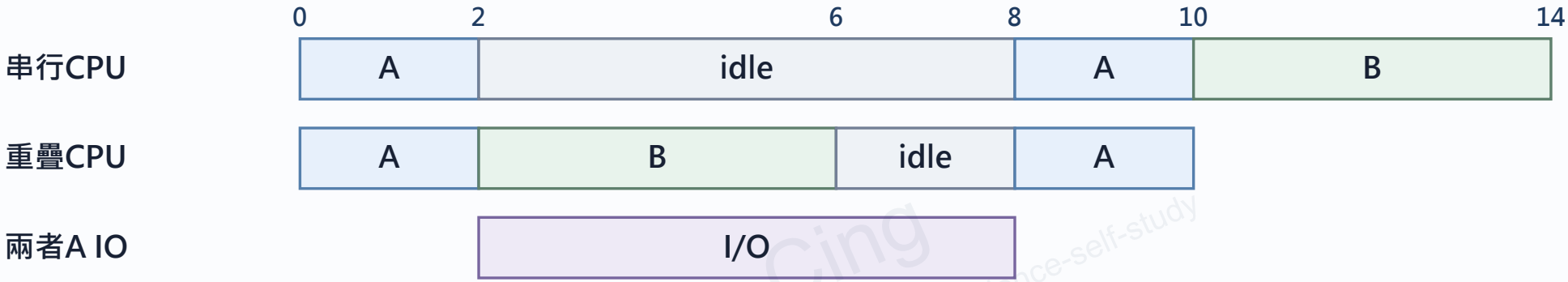
CPU總工作仍8，不是把同一CPU同時跑A/B。節省的4正是B與A的I/O重疊；若沒有其他ready工作或I/O都等待同一瓶頸，不會憑空省時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

multiprogramming是把I/O等待與別人的CPU工作重疊 (續)

同一工作量：A=CPU2→IO6→CPU2，B=CPU4

單CPU / 獨立IO、切換免費；藍=A CPU、綠=B CPU、灰=idle、紫=A I/O。



CPU總工作都8；串行總14、利用率4/7；重疊總10、利用率0.8，節省B與IO重疊的4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

time sharing再要求可回收控制權與可接受response

multiprogramming可在阻塞時換工作；若某程式CPU loop永不阻塞，單靠它自願讓出不能保證別人有進度。time sharing通常用timer及preemption，搭配ready queue給互動工作機會。

簡化模型有n個持續ready工作、每次最多跑q且每次切換成本c；固定RR輪轉，某工作剛錯過輪次至下次執行，等待上界為(n−1)q+n c（以包含切換成本的保守邊界計）。若n無界增加、kernel長時間不可搶占或更高priority可無限插入，此界不成立。

q小可降低輪轉等待但增加切換比例c/(q+c)。例q=4ms,c=0.1ms，純CPU輪轉的切換占0.1/4.1≈2.44%；這是固定一段q加一次c的模型，不是所有系統的實測開銷。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.5 作業系統運作

- user/kernel權限邊界由受保護入口與狀態強制。
- mode、thread與address-space切換不是同一事件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

dual mode與受保護入口形成執行邊界

一般硬體以mode／privilege state區分允許的操作。user不能直接改timer、page-table root或中斷入口等受保護狀態。system call依ISA保存必要返回狀態並轉到受控kernel entry；kernel檢查參數／authority後才執行服務。

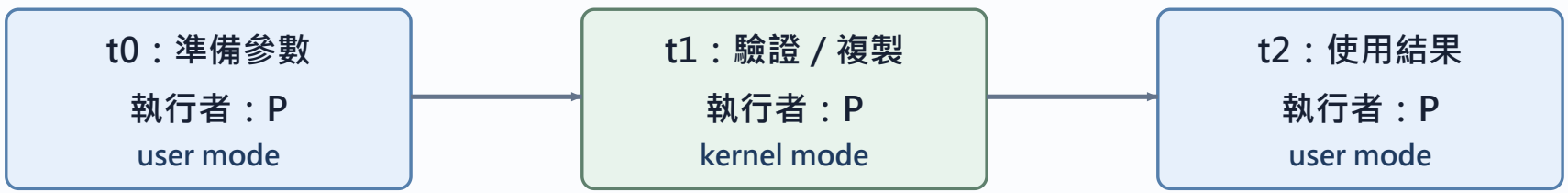
t	同一thread的執行	mode	必然換process？
0	user準備read參數	user	否
1	system call入口與驗證	kernel	否
2a	cache hit，複製並返回	kernel→user	否
2b	cache miss，等待I/O	kernel阻塞	可換到另一ready thread

mode switch只說特權狀態變化；context switch另保存／載入執行者。若換到同address space的另一thread，還不能因此推得address-space switch。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

dual mode與受保護入口形成執行邊界 （續）

cache-hit system call：兩次mode transition，零次thread切換



藍=user，綠=kernel。受控入口與返回改變mode；P的身分與address space不變。

本例無其他事件，直接返回P；P阻塞或被搶占時可排程Q。三種切換分開計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

timer保證的是回到受控機制，公平性還需scheduler

OS在交CPU給不可信user前設定timer；倒數到期產生受硬體控制的interrupt。若user能自行關閉timer或修改counter，就能破壞回收CPU的機制，因此相關操作須受保護。

有限進度條件：timer能被服務、不可搶占kernel段落有上界、handler與scheduler會終止、ready規則不讓同一工作永久飢餓。缺其中一條，只有timer硬體不保證每個process有CPU。

中斷可能延後、合批或受優先權控制，不能把設定q說成每個指令邊界必在q精準切走。核心也會發生fault與惡意參數攻擊；mode只是enforcement基礎，不證明所有kernel行為正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.6 程序管理

- process帶有資源與執行狀態，program只是被動內容。
- I/O完成先成ready，共享更新的多步交錯可遺失結果。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

program是被動內容，process是帶狀態的執行

program bytes可存在file；process有address space、credentials、open references與至少一個thread等狀態。thread保存PC、registers、stack等執行context；同process多thread共享部分資源，卻不是共享同一PC。

事件	狀態轉移	要由誰完成
等待I/O	running→waiting	當前thread／kernel登記等待
I/O完成	waiting→ready	完成路徑使其可被排程
scheduler選取	ready→running	scheduler／dispatcher
quantum到期	running→ready	受控preemption

I/O完成不表示立即running，仍要等CPU。把waiting也算成ready-queue waiting會混淆第6章的時間指標。程序管理還包括建立／終止、同步／IPC及資源清理，不只選下一個PID。

為何會有競爭：一個increment有多個步驟

共享counter=0，兩thread各做一次「load→+1→store」，不視為原子。

步	T1	T2	counter
1	load r1=0	—	0
2	—	load r2=0	0
3	store r1+1=1	—	1
4	—	store r2+1=1	1

預期2卻得到1。即使單CPU，若可在兩個步驟間切換也能發生；多CPU只是增加交錯方式。mutex或符合需要的atomic RMW能建立另一個不可分割規格，但volatile標記本身不等於互斥。完整memory ordering與同步條件在第5章展開。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

1.7 記憶體管理

- 相同VA可映射不同frame，授權與駐留分開判斷。
- 回收需要所有引用與映射失效，不只看到process釋放。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

address space把程式位址與實體frame分開

固定page size4096B，P的VPN1→PFN7、read/write；Q的VPN1→PFN9、read-only。相同VA=0x1234，VPN1、offset0x234=564；P對應PA0x7234，Q對應PA0x9234。

操作	P	Q
read VA0x1234	讀frame7+564	讀frame9+564
write VA0x1234	權限允許	permission fault
改自己的page table為frame7	一般user不能直接改受保護表	同樣受限制

映射存在、駐留與授權是不同檢查：not-present可能可由OS補頁；禁止寫入不一定是補頁就解決。shared memory則是OS明確讓多個映射指同一frame，並另定權限，不是VA相同自動共享。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

記憶體管理要維持的集合關係

在任一時點，已配置physical frames與可回收free frames不得同時宣稱獨占同一frame；有合法共享時，references與權限須一致。回收frame前還要確定舊映射不再可被使用，包括TLB與未完成I/O等持有者。

例RAM8 frames，kernel2、P3、Q2，free1。若Q釋放1且無其他reference，free變2；若該frame仍被P共享，就不能直接加進free。OS追蹤「誰仍能到達」比只數process宣稱free更重要。

virtual memory允許非所有virtual pages同時駐留，但不是無限RAM；working set、storage I/O與commit限制會影響效能／是否能履行配置。第8/9章給頁表與replacement的一般機制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.8 儲存管理

- cache的miss penalty是總成本或額外成本必須先定義。
- 配置bitmap、crash一致性與資料保護各有不同機制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

cache層次：latency模型要說明miss penalty的意義

固定每次先查L1花1cycle，hit率0.9，miss額外花50cycles，無重疊／prefetch：平均=1+0.1×50=6cycles。等價列法0.9×1+0.1×51=6。

若「50」指miss的總latency而非額外成本，則應是0.9×1+0.1×50=5.9；兩個輸入契約不同，不能只套同字母公式。平均也不代表每次都是6cycles，更不給tail latency。

cache將常用資料放快層，但引入副本一致性與失效問題。CPU cache coherence處理共享位址的快取副本，不等於filesystem durability；資料可能仍只在volatile層。write完成、flush完成與斷電後存在需分別界定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-stu

file、block allocation與儲存故障的不同責任

應用以file名稱／offset操作；filesystem把範圍映射到blocks，block layer／driver再送裝置。allocation要防同block被兩個獨占owner拿走，free-space索引必須與物件指標一致；memory lock只解同時競爭，不解crash途中只持久一半的更新。

例裝置2^20個可配置block，以1bit/block bitmap記free狀態，需2^20bits=2^17B=128KiB。這是logical bitmap容量，不是一次query只需1bit實體I/O；未cache時仍需載入所在儲存block。

metadata毀損、裝置失效、誤刪與惡意存取需要不同機制。RAID可針對某些硬體失效，不能單獨防止誤刪；backup與protection也不能互相代替。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.9 保護與安全

- authentication和authorization分別回答身分與操作權利。
- least privilege依有效references及requested rights判定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

protection與security先分威脅及強制位置

protection研究已知主體如何被限制使用物件；security還涉及身分驗證、入侵、惡意程式與系統威脅。兩者互相依賴，不是把一個稱「內部」、另一個永遠稱「外部」就能涵蓋所有攻擊。

問題	所需資訊／機制	單靠它仍不足
誰在請求？	authentication、可信credentials	知道身分不等於授權
可操作哪個物件？	ACL／capability、access check	UI顯示不會強制API
通訊內容被竊聽？	符合威脅模型的加密與key管理	不能修好已被攻陷端點
同址映射越權？	MMU權限與受保護頁表	kernel漏洞仍可能繞過policy

需要把資料機密性、完整性、可用性分別定義；拒絕所有請求雖可阻止某些洩漏，也會完全破壞可用性。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

least privilege及授權時點

例影像轉檔程式只應讀輸入file與寫指定輸出，不應取得整個home directory的可寫authority。以已授權物件reference授予所需rights，比把全域路徑／root身分交給它更容易限制損害。

但「一般user」不是零權限，繼承的file descriptors、共享memory、network sockets都可能帶authority。反過來，撤掉某名稱的權限不必然讓所有既有handle失效；必須看建立reference時與後續操作時的契約。

一個完整判定需列主體credentials、物件身分、requested rights、政策與檢查時點。只看檔名、user顯示名稱或程式是否有特權圖示，都不足以推論實際權限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.10　核心資料結構

- 結構選擇同時考慮lookup、update、ordering及node生命期。
- hash最壞碰撞與BST全部子樹範圍都有可構造反例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料結構必須同時符合查詢、更新與生命期

結構	適合操作與成本	必要前提／限制
linked list	已知節點／前驅後插刪O(1)	找key最壞O(n)，pointer與cache成本
stack／FIFO queue	LIFO／FIFO；正確端點可O(1)	FIFO不是priority order
balanced search tree	ordered lookup／range、O(log n)	維持特定平衡不變量
hash table	指定hash模型下平均lookup O(1)	collision可使最壞O(n)，擴容有成本
bitmap	有限整數域membership	空間隨域大小，需原子claim

BST真正條件是每個node「全部left-subtree keys < node.key < 全部right-subtree keys」（本例相異鍵）；只檢查left-child≤right-child不夠。平衡樹高度是O(log n)，不能把任意樹精確稱只有log2 n層。

同一組key的collision與錯誤BST反例

hash h(k)=k mod4，插5、9、13都進bucket1；chaining依序存[5,9,13]，找13要3次key比對，找17也要3次才判不存在。把n個同餘keys放一起，最壞線性；「hash O(1)」須另有分布／負載或更強設計保證。

BST例 root10、left5、right20，卻把12放在5的右子。12>5滿足局部parent比較，但12在10的left subtree，不滿足12<10；按BST搜尋12會往right20再找不到。必須驗整個subtree範圍。

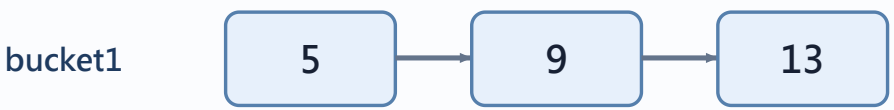
並行讀者可能仍持old node reference。更新者即使已從list/tree移除節點，也不能立刻free給別人重用。鎖、reference counting或RCU式grace period提供不同生命期契約；允許讀舊值不等於允許use-after-free。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一組key的collision與錯誤BST反例 (續)

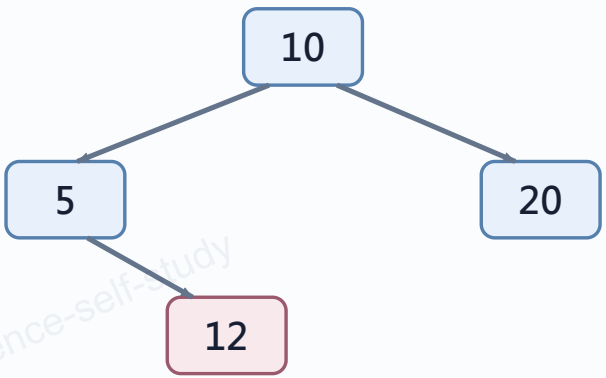
成本與不變量：hash碰撞、BST全部子樹範圍

h(k)=k mod4；chaining



找13：3次比較；找17：3次才判不存在。

n個同餘keys → 最壞O(n)。



紅12在root10的left subtree，卻12>10；違反全子樹範圍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.11 運算環境

- client/server、VM與distributed描述不同責任／隔離關係。
- real-time需檢查release、completion和deadline，不能只看平均。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

運算環境改變的是資源與失敗模型

環境	主要邊界	不能直接推論
client/server	client發請求、server提供服務	server一定不會也是別人的client
peer-to-peer	節點可同時請求／提供	完全無中央發現或驗證服務
cluster/distributed	獨立節點與network	timeout就證明remote已失效
virtual machine	guest OS經VMM使用虛擬硬體	guest權限等於host權限
container式環境	一般共享host kernel的隔離視圖	每個container都有獨立kernel
embedded／real-time	特定設備／時間約束	所有embedded都必須hard real-time

雲端的IaaS/PaaS/SaaS是提供與管理責任的層級，不是三種CPU架構；虛擬化可作基礎，但不是每個雲服務都等同一部VM。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

real-time的正確性包含deadline，而非只看平均快

固定單CPU、無阻塞／overhead，週期工作A(C=1,T=D=4)、B(C=2,T=D=6)。CPU demand率1/4+2/6=7/12；只表示負載小於1，沒有指定scheduler仍不能直接保證每個deadline。

一個可行的12單位EDF排程：A0–1、B1–3、idle3–4、A4–5、idle5–6、B6–8、A8–9、idle9–12；A在4/8/12前完成，B在6/12前完成。12後同步重複，因period的lcm=12且邊界無未完成工作。

若加入非搶占critical section、release jitter或I/O等待，這個schedule證明的前提已變，需重新算response／blocking。低平均latency不能替代最壞deadline檢查；hard/soft real-time區別在違約後果及允許程度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

1.12 開源作業系統

- open source是授權和來源可用性，不自動保證品質。
- API契約、固定版本source與工作負載測量是不同證據。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

open source是授權與來源可用性，不是品質保證

依Open Source Definition，僅能看到source還不夠；授權還須允許符合定義的散布與衍生等權利。source available、freeware、public domain、open source不是同義詞，商業收費也不自動與open source衝突。OSI定義。

以Linux kernel搭配工具／library／服務／套件形成distribution，可區分kernel與整個可使用環境；BSD也是另一種系統家族與授權傳統。不能因某component開放就推論整個distribution每一檔同授權。

source能讓人核讀實作與重建，但需固定版本、設定與相依元件才能比較行為。原書的Linux3.2、Windows7等案例保留歷史契約；當代文件只在明確標示的補充介面使用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從介面到原始碼：如何判斷一項系統保證

要問的事	應查的證據	仍不能用什麼取代
API允許什麼結果？	該版本的介面／標準契約	只看一次成功執行
特定版本怎麼做到？	對應source、build configuration與架構	只看其他版本同名function
觀察的latency是什麼？	同工作負載的時間與資源分解	把operations比例當時間比例
更新是否持久？	儲存／flush契約及故障次序	只見write回傳成功

開源使source可查，並不自動證明沒有race、最佳效能或安全。對照算法與具體資料時仍要保留前提、錯誤結果及一般不變量；只用一個代表性範例無法推出所有輸入都成立。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

一個read請求如何連起全課程

P用已授權fd讀取file的4096B：先從user進受控kernel入口；查descriptor/OFD與buffer；cache miss時建立I/O、P等待；scheduler選Q；DMA完成後把P設ready；P再取得CPU、資料可見後複製／回傳實際bytes。

環節	要追的狀態	後續章節
呼叫與保護	mode、credentials、requested right	2、14、15
process／thread	ready/waiting/running、saved registers	3–7
映射與資料	VA/PTE/frame、cache、buffer lifetime	8、9、13
名稱與儲存	OFD offset、inode、block、完成／durable	10–12

這些state不在同一瞬間全部變成「完成」。尤其waiting→ready不等於立即running，request完成不等於其他file已flush。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最容易混淆的四組量

- mode transition／thread context switch／address-space switch分開計；同thread的cache-hit system call可只有前者。
- CPU busy時間、request latency、throughput、deadline各有分母；CPU省工不必然讓單次latency下降。
- 名稱、物件身分、已開reference分開；修改路徑不必改變既有fd。
- 平均成本、最壞成本、一般正確性分開；hash碰撞、timer延後與shared-memory競爭都有合法反例。

先固定系統版本與抽象、列可觀察結果和狀態轉移，再用不變量／反例判斷機制。這些共同語言支撐後續章節的形式推理，而不是每遇新系統就重新猜同一個名詞。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

參考資料

- Silberschatz、Galvin、Gagne，Operating System Concepts，第9版，第1章，印刷頁3–52。本文機制與章節架構參考該章，數例、反例、練習和SVG為原創。實際圖核讀：Fig1.3（8）interrupt timeline、Fig1.9（19）multiprogramming配置。
- 原書BST／hash的概括敘述在此以完整子樹不變量與collision前提表達，不將一般hash保證成最壞O(1)。core/socket/hardware thread、SMP/NUMA及mode/context各自分開。
- Open Source Initiative：Open Source Definition：開源與僅source可見不同。授權條件以各component實際license為準。
- 時間、機率與記憶體數例均附模型前提；DMA與排程時間是教學給定值，不是特定硬體量測。歷史版本細節留在後續對應系統案例。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study